

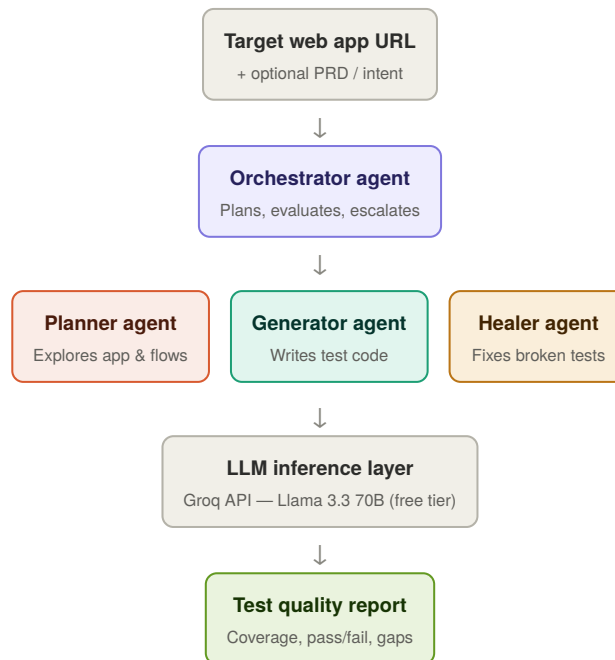
Autonomous Test Orchestration Agent

End-to-end, free-tools build guide, architecture, and differentiation strategy

Prepared for: Aivar Innovations submission · September 2026

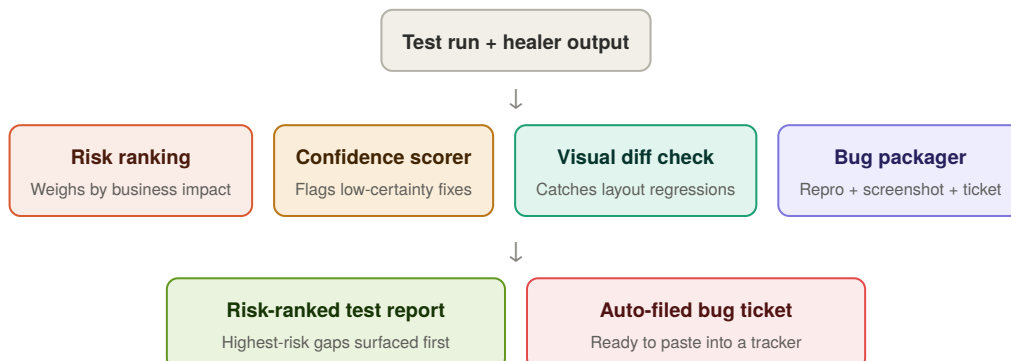
1. Core pipeline architecture

The base pipeline every team is expected to build, per the problem statement:



2. Differentiation engine — where you actually win

Building only the diagram above puts you at parity with every competent team. The judged 20% "innovation" score is decided by what sits on top of it. This is the layer that turns raw Healer/test output into decisions and artifacts a developer can act on immediately:



Why each module matters

Module	What it does	Why judges notice it
Risk ranking	Classifies each discovered flow by business impact (checkout/payment/auth > wishlist/footer) and orders the whole report by risk, not by flow count.	Turns a flat pass/fail list into a prioritized decision tool — this is the "business impact" score, made concrete.
Confidence scorer	Every Healer fix and every defect classification carries a confidence value. Low-confidence actions are flagged for human review instead of silently applied.	Shows the system knows what it doesn't know — a rare, senior-engineer signal in a hackathon demo.
Visual diff check	Lightweight screenshot comparison against a baseline, catching layout breaks (overlap, off-screen elements) that pure DOM/selector checks miss.	Catches a whole class of real bugs competitors structurally can't see.
Bug packager	For every confirmed defect, auto-generates a minimal repro, a screenshot, and a ticket-ready title/description.	Closes the loop from "found a bug" to "filed a bug" — the single most immediately useful output for a developer.

Stretch differentiators (build a thin version, pitch the rest as roadmap): a regression radar that diffs today's run against a stored baseline from an earlier run ("3 flows changed behavior since last time"); a visible cost-aware model-routing dashboard showing cheap-model vs reasoning-model spend per decision; a small pattern library that carries lessons learned across different target apps (e.g. "always test cart persistence across a session refresh" for e-commerce sites).

3. Free tools — full stack

Layer	Tool	Why	Cost
Orchestration framework	LangGraph (or CrewAI)	Stateful agent graph with conditional edges for re-plan / escalate / continue	Free, open-source
LLM inference	Groq API	No-card free tier, fastest inference on the market — visible speed advantage live in a demo	Free tier
Browser automation	Playwright (Python)	Generates selectors, headless, works against any target app	Free
App exploration	Playwright BFS crawler or browser-use (OSS)	Lets the Planner "see" the app instead of hardcoded flows	Free
Visual diff	Playwright screenshot + Pillow / pixelmatch-py	Simple pixel-diff baseline comparison, no paid service needed	Free
Vector store (PRD gap analysis, pattern library)	ChromaDB (local, in-memory)	Embed PRD/plan sections and compare; also backs the stretch pattern library	Free
Backend/API	FastAPI	Async-friendly for orchestrating agent calls	Free
Demo frontend	Streamlit	Fast to build a live pipeline/decision dashboard	Free
Hosting for demo	Localhost + ngrok, or Render/ Railway free tier	Only needs to be live for the judged demo	Free
Repo & diagrams	GitHub + Mermaid (renders natively in README)	Required submission deliverables	Free
Fallback LLM (optional)	Google Gemini free tier / GitHub Models	Auto-switch on a Groq 429 so a demo never stalls on rate limits	Free

About Groq specifically

- No credit card required — sign up at console.groq.com and start calling the API within minutes.
- Rate limits apply at the organization level and per model, not per user — spreading calls across two models effectively doubles your usable daily budget.
- Representative free-tier limits on current production models (verify against your account's limits page before the event): GPT-OSS 120B / 20B — 30 requests/min, 1,000 requests/day, 8,000 tokens/min, 200,000 tokens/day. Llama 3.3 70B Versatile — 30 requests/min, 1,000 requests/day, 12,000 tokens/min, 100,000 tokens/day.
- Speed: roughly 1,000 tokens/second on GPT-OSS 20B and 280–394 tokens/second on Llama 3.3 70B, versus 50–100 TPS typical of GPU-based cloud APIs elsewhere — a genuine, visible demo advantage.
- **Recommended split:** Llama 3.3 70B for judgment calls (coverage evaluation, risk ranking, defect classification); a smaller/faster model (Llama 3.1 8B Instant or GPT-OSS 20B) for mechanical plan-to-code generation, to conserve the free-tier quota.

4. Step-by-step build order

1. **Set up accounts & environment.** Groq API key, pick your own stable test target now (don't wait for the organizer's URL), install LangGraph, Playwright, FastAPI, Streamlit, ChromaDB.
2. **Build the skeleton state graph first** — wire Orchestrator → Planner → Evaluate → Generator → Run → Healer → Report with stub functions before writing any "smart" logic. Most teams lose points to a broken pipeline, not a dumb agent.
3. **Build the Planner** — crawl with Playwright, convert the structure into a JSON test plan (flow, steps, expected outcome, category: happy path / edge case / error state). Explicitly prompt for non-happy-path coverage.
4. **Build the coverage-evaluation gate** — a rubric-driven LLM check that routes back to the Planner with specific feedback when a flow is missing edge cases, error states, or auth coverage.
5. **Add risk ranking** right after the plan is approved — classify each flow's business impact and carry that weight through generation, healing, and the final report.
6. **Build the Generator** — convert plan steps into Playwright tests, validating every selector live against the running page before finalizing it.
7. **Build the Healer with confidence scoring** — on failure, gather the error trace + DOM snapshot, classify script-issue vs. real defect, and attach a confidence score; low-confidence outcomes get flagged for human review instead of auto-applied.
8. **Add the visual diff check** — a screenshot comparison run alongside the DOM-based test to catch layout regressions pure assertions miss.
9. **Add the bug packager** — for every confirmed defect, auto-generate a repro script, screenshot, and ticket-ready description.
10. **Build the report synthesis** — risk-ranked coverage, pass/fail, healer actions with reasoning, remaining gaps, and the packaged bug tickets.
11. **Wire the Streamlit demo UI** — show the agents' live decisions ("Planner found 12 flows", "gap detected, re-planning", "Healer: 40% confidence, flagging for review"), not just a spinner and a final report.
12. **Stretch goals, time permitting** — PRD-to-plan gap analysis, parallel execution across flows, the regression-radar / pattern-library ideas as a pitched roadmap slide even if only partly built.
13. **Package deliverables** — README with setup + Mermaid architecture diagram, 2–5 min demo video, slide deck covering problem, approach, trade-offs, business impact.

5. Scoring map — what actually moves each criterion

Criterion	Weight	What moves it
Functionality & completeness	30%	Runs end to end with zero manual steps on your own target <i>and</i> the organizer's target. Rehearse both.
Innovation & originality	20%	The coverage gate, risk ranking, confidence scoring, and defect classification — the decision-making layer, not the tech stack.
Technical implementation & code quality	20%	Clean state graph, real error handling, Healer that inspects evidence rather than blindly re-running.
UX & demo clarity	15%	Show agent reasoning live in the dashboard log, not a black box that only shows a final report.
Business impact & feasibility	10%	Risk-ranked output and auto-filed bug tickets give you a concrete, quotable "this saves a QA team N hours" story.
Presentation	5%	Name your differentiators explicitly and be honest about trade-offs and scope cuts.

Avoid: CI/CD integration, cross-browser matrices, and any manually written test script — all three are explicitly out of scope in the problem statement and will not earn points, and hand-writing a test to "make sure the demo works" directly violates a Must Have requirement.

6. The prompt — feed this to GPT (or Claude/Cursor) to build it end to end

This version bakes the differentiators in as required features, not afterthoughts, so the scaffold you get back is already competitive, not generic.

You are acting as a senior AI engineer building a hackathon submission. Build a complete, runnable, end-to-end "Autonomous Test Orchestration Agent" system to the exact specification below. Produce production-quality code, not a toy demo — proper error handling, logging, config management, and a clean repo structure I could hand to another engineer.

CONTEXT / PROBLEM STATEMENT

Build an autonomous test orchestration agent that takes a web application URL as input and drives the full testing lifecycle — planning, test generation, execution, and repair — without human intervention between stages. Coordinate a pipeline of three specialised sub-agents:

1. Planner — explores the target application and produces a structured, human-readable test plan covering meaningful user flows, not just happy paths (must include edge cases and error states).
2. Generator — converts the plan into executable test code with live selector and assertion validation against the running app (no hallucinated selectors).
3. Healer — replays failing tests, inspects the failure evidence, and repairs broken locators/flows OR classifies the failure as a genuine application defect (must not "fix" a real bug by weakening the test).

A meta-agent (Orchestrator) coordinates this pipeline: evaluates the Planner's coverage quality BEFORE generation and sends it back for revision if there are gaps; decides when to re-plan vs proceed vs escalate; and synthesizes all sub-agent outputs into one final report.

DIFFERENTIATION LAYER — implement these as first-class, required features, not bolt-ons:

1. Risk-weighted coverage: after the Planner's plan is approved, classify every discovered flow by business impact (e.g. checkout/payment/auth = high, informational/cosmetic = low) using an explicit rubric you define, and carry that weight through generation, execution, and the final report. The final report must be ORDERED by risk, not by flow index or pass/fail status.
2. Confidence-scored decisions: every Healer action (auto-patch vs flag-as-defect) and every Orchestrator escalation decision must carry a numeric confidence score (0-1) with a stated rationale. Define a threshold (e.g. below 0.6) below which the action is NOT auto-applied — instead it's queued in the report as "needs human review" with the evidence attached. This must be visibly different behavior in the code and in the report output, not just a logged number.
3. Reproducible bug packaging: for every failure the Healer classifies as a genuine application defect (not a script issue), auto-generate: (a) a minimal standalone repro script derived from the failing test, (b) a screenshot at the point of failure, (c) a ticket-ready title and description in plain English suitable for pasting into GitHub Issues/Jira. Output these as a distinct artifact per bug, not just a line in the report.
4. Visual regression check: alongside each test's DOM/assertion checks, capture a screenshot and do a simple pixel-diff against a stored baseline screenshot (first successful run = baseline). Surface any visual diff above a configurable threshold as a separate finding category from functional failures, since a test can pass functionally while the layout is visibly broken.

Good-to-have (implement behind a clean feature flag if time allows):

- Accept an optional PRD document; inform the Planner's scope with it, and after generation, do a PRD-to-test-plan gap analysis showing which stated requirements are not covered.
- Accept natural-language user intent (e.g. "focus on checkout and authentication") and bias the Planner's scope accordingly.
- Parallel test execution across independent flows.
- A "regression radar" mode: store the plan + results from a run as a baseline, and on a later run against the same or a different URL for the same app, diff behavior against that baseline and report what changed.

Explicitly out of scope — do not build: production-scale deployment/hosting, CI/CD pipeline integration, cross-browser test matrices, complete coverage of a production app, any manually written test script (100% of test behavior must originate from the agent pipeline).

CONSTRAINTS

- Free tools only, no paid API keys. Use Groq (<https://console.groq.com>) as the LLM provider via its OpenAI-compatible chat completions endpoint, reading GROQ_API_KEY from an environment variable — never hardcode it. Use Llama 3.3 70B for reasoning-heavy calls (coverage evaluation, risk classification, confidence scoring, defect classification) and a smaller/faster Groq model (e.g. Llama 3.1 8B Instant) for mechanical plan-to-code generation, to conserve free-tier rate limits, which apply per-model and per-organization.
- Python. LangGraph for the agent orchestration state machine (explicit nodes and conditional edges, not a single monolithic prompt chain). Playwright (Python, async API) for all browser automation: crawling, live selector validation, test execution, and screenshot capture for the visual diff module.
- Add a retry/fallback layer for LLM calls: exponential backoff on 429s, and structure the LLM client as a small provider-agnostic abstraction so a second free provider (e.g. Google Gemini free tier) could be added later as a fallback without refactoring callers.

- FastAPI backend exposing: POST /run (starts a pipeline run given a URL, optional PRD text, optional intent), GET /run/{id}/status (current stage + agent decisions, including confidence scores and risk classifications as they're produced), GET /run/{id}/report (final report), GET /run/{id}/bugs/{bug_id} (a single packaged bug artifact).
- Minimal Streamlit frontend showing: input form (URL + optional PRD upload + intent text box); a live-updating decision log (not just a spinner) that visibly shows coverage-gate re-plans, risk classifications, and low-confidence flags as they happen; a final report view with a risk-ranked table, a distinct visual-regression findings section, and a list of packaged bug tickets with their repro/screenshot attached.
- ChromaDB (local/in-memory) for the optional PRD-to-plan gap analysis — decoupled behind a feature flag so the core pipeline works without it.

DELIVERABLES — produce all of these

1. Full repo structure and every file's code (not snippets), organized roughly as:
 - /agents (orchestrator.py, planner.py, generator.py, healer.py)
 - /differentiation (risk_ranking.py, confidence_scorer.py, visual_diff.py, bug_packager.py)
 - /graph (LangGraph state machine definition, state schema)
 - /llm (Groq client wrapper with retry/fallback abstraction, prompt templates per agent)
 - /browser (Playwright crawler, selector validator, test runner, screenshot capture)
 - /api (FastAPI app)
 - /ui (Streamlit app)
 - /reports (report generation/rendering logic)
 - requirements.txt, .env.example, README.md
2. The full LangGraph state schema covering: target_url, prd_text, user_intent, test_plan, coverage_evaluation, risk_classifications, generated_tests, run_results, healer_actions (each with a confidence score and rationale), visual_diff_findings, packaged_bugs, final_report, and a re-plan loop counter capped at 2 (force-proceed with a noted risk after that).
3. Every prompt template, requiring structured JSON output with the schema specified in the prompt, and defensive JSON parsing around every LLM call (one retry with "your last output was invalid JSON, here is the error" on failure before hard-failing the node).
4. The full coverage-evaluation rubric and the risk-classification rubric, written out explicitly (what counts as high/medium/low business risk, and why), plus the conditional edge logic that routes back to the Planner with specific feedback on a coverage gap.
5. The full confidence-scoring and defect-classification logic in the Healer: what evidence it gathers (error trace, DOM snapshot at failure, expected vs. actual selector state), the exact prompt used, how the confidence score is derived, and the branching logic at the 0.6 threshold.
6. The full visual-diff and bug-packaging modules, including how baselines are stored and how a packaged bug artifact (repro script + screenshot + ticket text) is generated and returned via the API.
7. A working final report generator producing both a machine-readable JSON report and a rendered HTML/Markdown version, ordered by risk, with a distinct section for visual-regression findings and a list of packaged bugs.
8. A README.md with: setup instructions (including getting a free Groq API key and rate-limit-aware defaults), architecture explanation, a Mermaid diagram of the full pipeline including the differentiation layer, how to run against any target URL, and known limitations — including an honest note on which stretch features (PRD gap analysis, regression radar, parallel execution) are implemented vs. only scaffolded.
9. Unit-test-style sanity checks for the non-agent logic (JSON parsing, selector validation, risk rubric application, confidence threshold branching, report generation) — not agent behavior itself, since that's inherently non-deterministic — plus a note in the README on how you'd evaluate agent quality over time (e.g. a fixed rubric run against a fixed target app as a regression check).

Work through this systematically: first output the full repo structure and state schema, then implement each module in order (LLM layer → graph/orchestrator → Planner → Generator → Healer → differentiation layer (risk/confidence/visual-diff/bug-packager) → report → API → UI), explaining key design decisions briefly as you go. Flag any place where you had to make an assumption due to ambiguity in the spec, and explain the trade-off. Do not skip error handling or silently stub out any required feature — if something genuinely can't be completed, say so explicitly rather than pretending it's implemented.

Using this prompt: feed it to your coding assistant in a fresh session, then work through the output module by module rather than accepting one giant dump. Have it actually run the Planner's crawl logic against your specific chosen target app early — generic scaffolding breaks fastest against a real page's quirks, and you want that failure on day one, not the night before the demo.